

Destructuring

1. Object destructuring

You have:

```
const user = { name: "Amit", age: 25, city: "Delhi" };
```

Extract name and city and print them.

2. Array destructuring

```
const colors = ["red", "green", "blue", "black"];
```

Extract first two values into variables and store the remaining in another variable.

3. Nested destructuring

```
const user = {  
  name: "A",  
  address: { city: "Delhi", pincode: 110001 }  
};
```

Extract city.

4. Destructuring in function parameter

```
const student = { name: "A", marks: 90 };
```

Write a function that prints name and marks using destructuring.

Map

5. Add grace marks

```
const marks = [50, 60, 70, 80];
```

Add 5 marks to each student using map.

6. Convert array to objects

```
const names = ["A", "B", "C"];
```

Convert into:

```
[{name: "A"}, {name: "B"}, {name: "C"}]
```

7. Price update

```
const products = [  
  {name: "Shirt", price: 500},  
  {name: "Shoes", price: 1500}  
];
```

Increase price of each product by 10%.

Filter

8. Passing students

```
const marks = [35, 60, 20, 80, 45];
```

Return only marks greater than or equal to 40.

9. Even numbers

From an array, return only even numbers.

10. Expensive products

Filter products with price greater than 500.

Find

11. Find user by id

```
const users = [  
  {id: 1, name: "A"},  
  {id: 2, name: "B"},  
  {id: 3, name: "C"}  
];
```

Find the user with id = 2.

12. Find first match

Find the first number greater than 50 from an array.

Arrow Functions

13. Convert function

Convert:

```
function add(a, b) {  
  return a + b;  
}
```

into arrow function.

14. Implicit return

Write an arrow function that returns square of a number using implicit return.

Combined Scenarios

15. Student processing

```
const students = [  
  {name: "A", marks: 80},  
  {name: "B", marks: 35},  
  {name: "C", marks: 60}  
];
```

Tasks:

1. Get students with marks ≥ 40
 2. Increase their marks by 5
 3. Return only names
-

16. Cart system

```
const cart = [  
  {name: "Shirt", price: 500},  
  {name: "Shoes", price: 1500},  
  {name: "Cap", price: 200}  
];
```

Tasks:

1. Get total price
2. Get products with price > 500

3. Get only product names

17. Filter + Map

From an array:

- Filter numbers greater than 10
 - Then square them
-

18. Find + Destructuring

Find a user and directly extract their name using destructuring.